

Large Language Models on HPC

Day 3

Dr. Rachel DuBose

University Libraries Research Computing Services

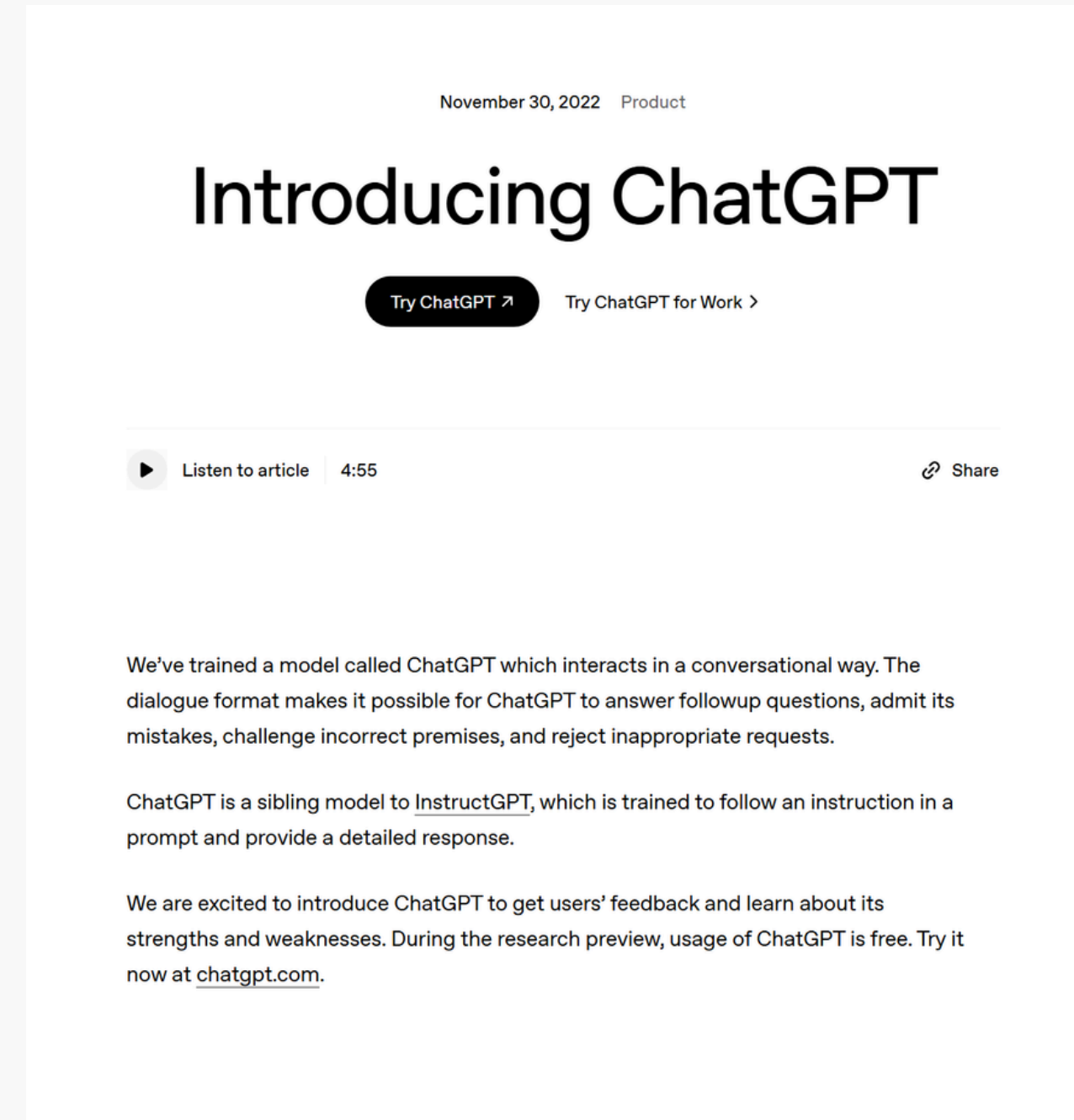
Overview

- Conceptual overview of large language models (LLMs)
- Considerations, ethics, and responsible use
- Containers
- Interactive jobs
- Building your first container on UAHPC
- Container best practices

Large Language Models (LLMs)

Large Language Models (LLMs)

process and generate text with coherent communication and capability to generalize to multiple tasks



Knowledge cut off

AI Knowledge Cutoff

The AI knowledge cutoff is a specific date when an AI model stopped learning from its training data. This means the AI cannot know about world events, news, or new technologies that happened after that fixed point in time. The Freshness Score reflects the knowledge cutoff date's relative position on a 100-point scale between 2021 and now.

Data sourced from [OpenRouter API](#) and manual data collection | Last updated: Jun 23, 2026, 7:41 PM UTC

Q Search...

Suggestions

Anthropic

Google

OpenAI

xAI

2026 Knowledge Cutoff AI Models

Model Name	Knowledge Cutoff	Freshness Score
> Anthropic: Claude Opus 4.7	2026-01-31	93

2025 Knowledge Cutoff AI Models

Model Name	Knowledge Cutoff	Freshness Scc
> xAI: Grok 4.3	2025-12-31	91
> OpenAI: GPT-5.5 Pro	2025-12-01	90
> OpenAI: GPT-5.5	2025-12-01	90
> Qwen: Qwen3 Next 80B A3B Thinking	2025-09-30	87
> Qwen: Qwen3 Next 80B A3B Instruct	2025-09-30	87

Each model has a knowledge cut off..

the way a model responds to questions about things and events beyond its knowledge cutoff may vary

Microsoft: Phi 4 Knowledge Cutoff Date

2024-06-30

Freshness Score

64

Microsoft: Phi 4

[Microsoft Research](/microsoft) Phi-4 is designed to perform well in complex reasoning tasks and can operate efficiently in situations with limited memory or where quick responses are needed. At 14 billion...

Tokens in LLMs

A **token** is the smallest unit of text that a model processes.

LLMs operate on a maximum token limit, which includes both the input (prompt) and the output (generated text).

FEATURE	 Phi 4	 GPT OSS 20B	 GPT OSS 120B
GENERAL INFO			
Context Window	Input 16,000 tokens	Input 131,072 tokens	Input 131,072 tokens
	Output 16,000 tokens	Output 32,768 tokens	Output 131,072 tokens

<https://llm-stats.com/models/compare/phi-4-vs-gpt-oss-20b-vs-gpt-oss-120b>

Each model has a context size

The **context size** is how many tokens the model can consider at once.

With a “too small” context window

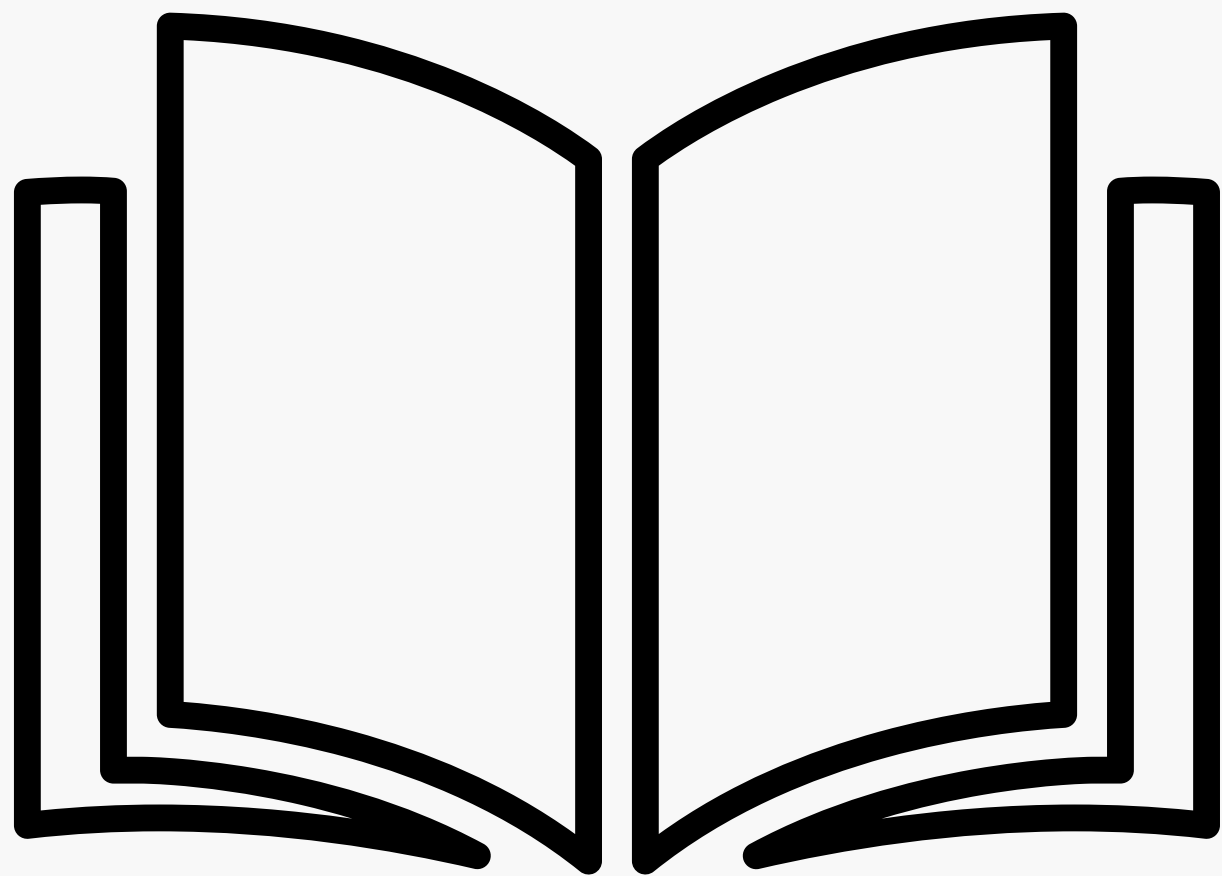
- model can forget instructions
- can reduce output coherence
- model unable to handle complex tasks

With a larger context window

- can increase computational cost
- potentially slower inference speeds
- attention dilution can occur

There is a current push to extend the boundaries to 10M

The model card has much of this info



*highly
recommended
reading!*

Phi-4 Model Card	
Phi-4 Technical Report	
Model Summary	
Developers	Microsoft Research
Description	phi-4 is a state-of-the-art open model built upon a blend of synthetic datasets, data from filtered public domain websites, and acquired academic books and Q&A datasets. The goal of this approach was to ensure that small capable models were trained with data focused on high quality and advanced reasoning. phi-4 underwent a rigorous enhancement and alignment process, incorporating both supervised fine-tuning and direct preference optimization to ensure precise instruction adherence and robust safety measures
Architecture	14B parameters, dense decoder-only Transformer model
Inputs	Text, best suited for prompts in the chat format
Context length	16K tokens
GPUs	1920 H100-80G
Training time	21 days
Training data	9.8T tokens
Outputs	Generated text in response to input
Dates	October 2024 – November 2024
Status	Static model trained on an offline dataset with cutoff dates of June 2024 and earlier for publicly available data
Release date	December 12, 2024
License	MIT

<https://huggingface.co/microsoft/phi-4>

Considerations, ethics, and responsible use

Use only AI tools approved by OIT. OIT has approved **locally run LLMs used for training, research, or instruction**.

Always follow OIT guidelines and minimum AI security standards.
Sensitive or restricted data should not be used with generative models.

Students should check course policies before using AI. When conducting research, speak with your research advisor before using AI tools. Use language models responsibly.

Consider first if your use-case is appropriate. Users should always verify outputs for accuracy and bias. Attribute any usage of AI or language models clearly, indicating that the output is machine-generated; and remember that language models can be helpful AI assistants, but they are computational tools, not humans.

More resources

<https://www.microsoft.com/en-us/ai/responsible-ai>

<https://ai.ua.edu/>

<https://uateachingacademy.ua.edu/ai-teaching-network/>

<https://ai.eng.ua.edu/>

Containers

A **container** is a self-contained, portable, and reproducible software runtime instance based on an **image**.

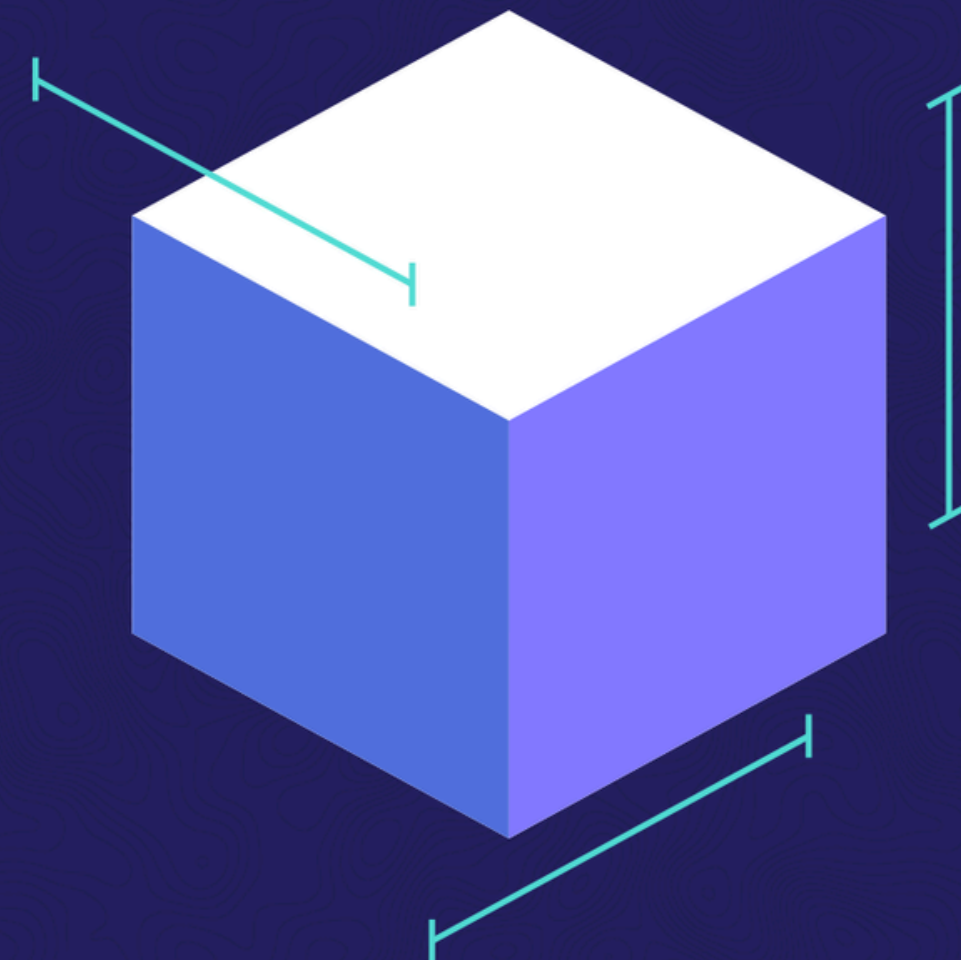
An **image** is a bundle of files that includes an operating system and software. It can also include data and other application-related files.

Open Container Initiative

The **Open Container Initiative** is an open governance structure for the express purpose of creating open industry standards around container formats and runtimes.

Established in June 2015 by Docker and other leaders in the container industry, the OCI currently contains three specifications: the Runtime Specification (runtime-spec), the Image Specification (image-spec) and the Distribution Specification (distribution-spec). The Runtime Specification outlines how to run a “filesystem bundle” that is unpacked on disk. At a high-level an OCI implementation would download an OCI Image then unpack that image into an OCI Runtime filesystem bundle. At this point the OCI Runtime Bundle would be run by an OCI Runtime.

[Learn more ↗](#)



Apptainer

Apptainer is a **container platform** that allows software engineers and researchers to easily share their work with others by packaging and deploying their software applications in a portable and reproducible manner.

Apptainer was created to run complex applications on HPC clusters.



Apptainer

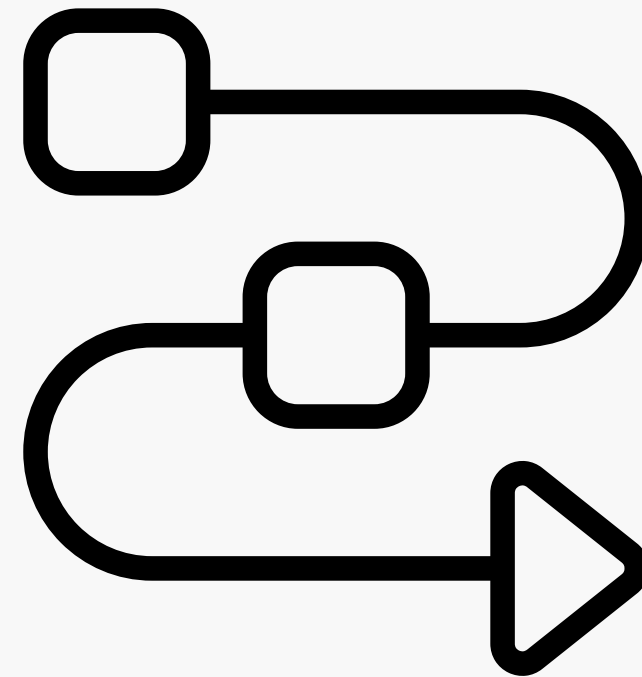
vs.

Docker

- Apptainer does not require root privileges.
- Apptainer uses SIF as its default container format. A SIF container is a single file, which makes it easy to manage and distribute.
- The container filesystem is read-only by default, yielding immutable images that can be signed.
- Is optimized for schedulers like Slurm.

- Docker requires root privileges.
- Docker uses a layered image format that is mutable. This brings convenience for iterative development but is less ideal for reproducible HPC workloads.
- Container filesystem is read/write by default.
- Docker has deep integration with Kubernetes and Docker Swarm.

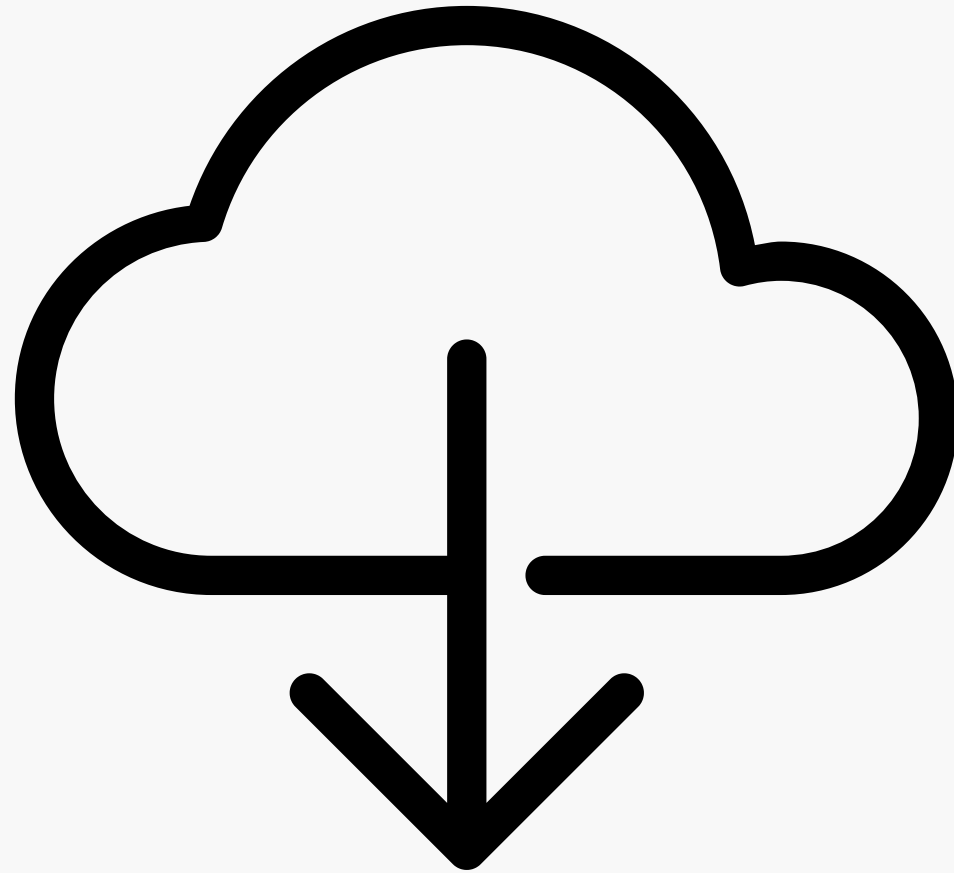
Container workflow



1. Define the container
2. Build the container
3. Execute/Run the container

Options to define containers

Pull



from Docker Hub,
Quay, or GHCR

Create from scratch



.def file

Interactive jobs

Interactive jobs allow users to monitor and interact with applications in real-time.

On HPC systems, interactive jobs are often required to build containers.

Why?

Long build times: Container builds can take hours.

Resource needs: Building may require significant CPU, memory, or disk space.

Dependency resolution: Some packages need to be installed in a specific order or with special flags.

Interactive jobs

We will use the **srun** command.

```
srun --pty --partition=main --qos=main --mem=32G --cpus-per-task=16 --ntasks=1 --time=02:00:00 bash  
srun --pty --gpus=1 --partition=gpu --qos=gpu --mem=32G --cpus-per-task=16 --time=01:00:00 bash
```

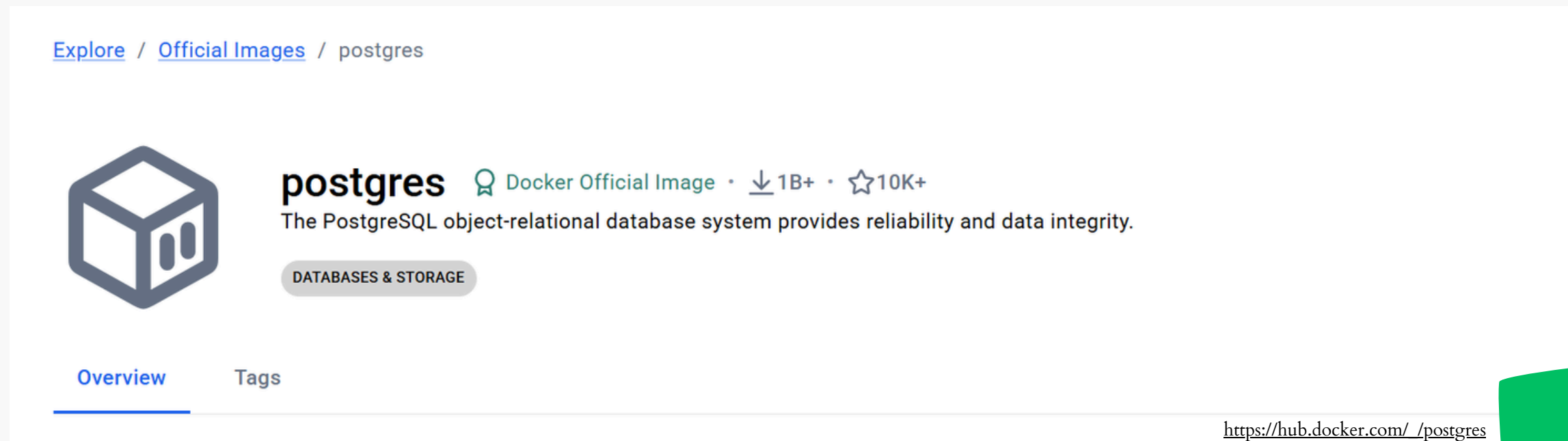
salloc can also be used; it is similar to **srun** except it only allocates the resources.

The user should type **exit** when finished using the node to release the allocation.

How to pull containers

Apptainer has the ability to convert available docker images into sif format allowing them to be run on HPC.

```
apptainer pull <local_image_name>.sif docker://docker_image.
```



```
ssh myBamaID@uahpc.ua.edu      then authenticate
```

```
module load apptainer
```

```
srun --pty --partition=main --qos=main --mem=32G --cpus-per-task=1 --ntasks=1 --time=30:00 bash
```

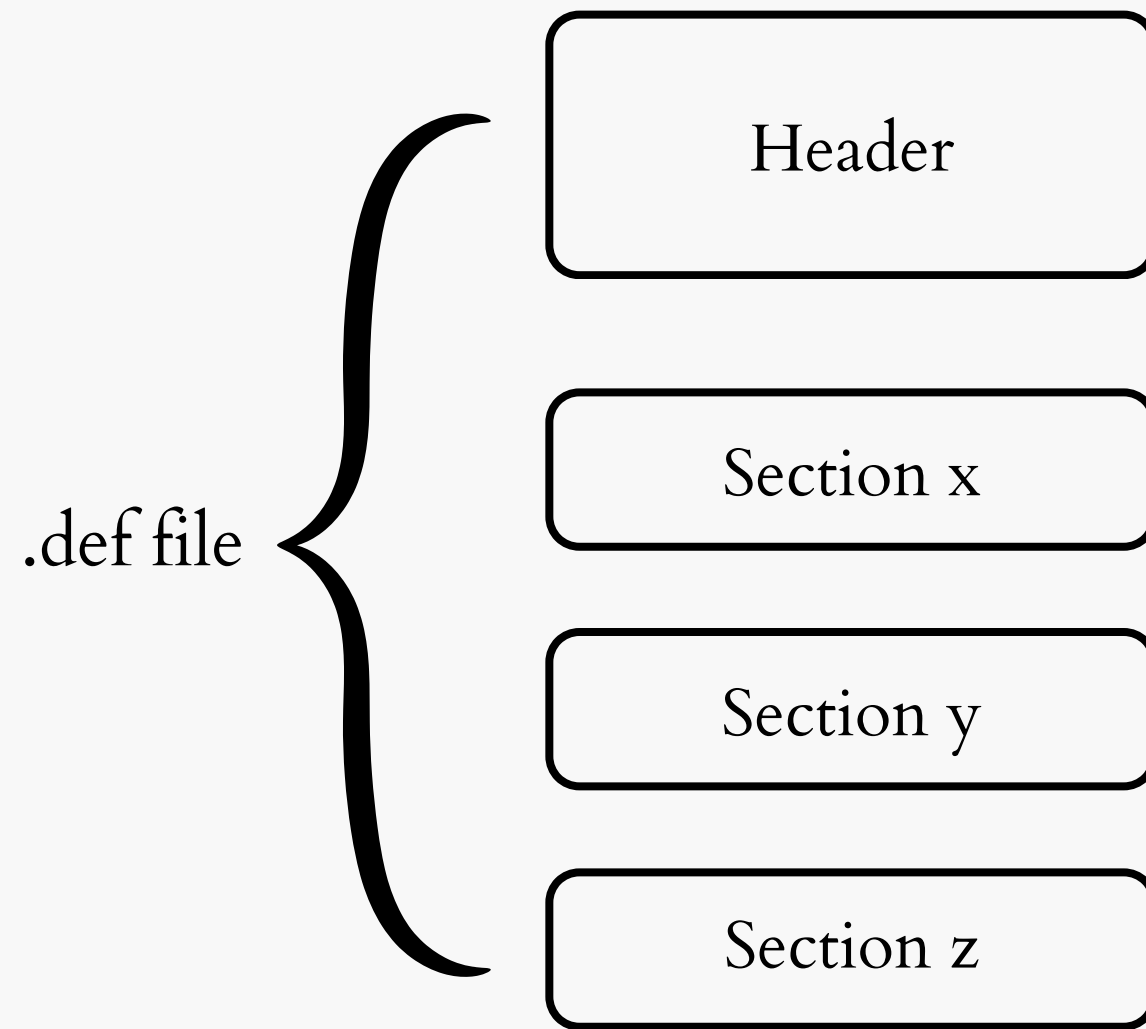
```
apptainer pull postgres.sif docker://postgres:15.18-bookworm
```

```
ls
```

No issues

Help!

How to build containers using a .def file



Header

Required keyword: Bootstrap — must be the first entry in the header.

Purpose: Tells Apptainer which bootstrap agent to use to get the base OS

Example:

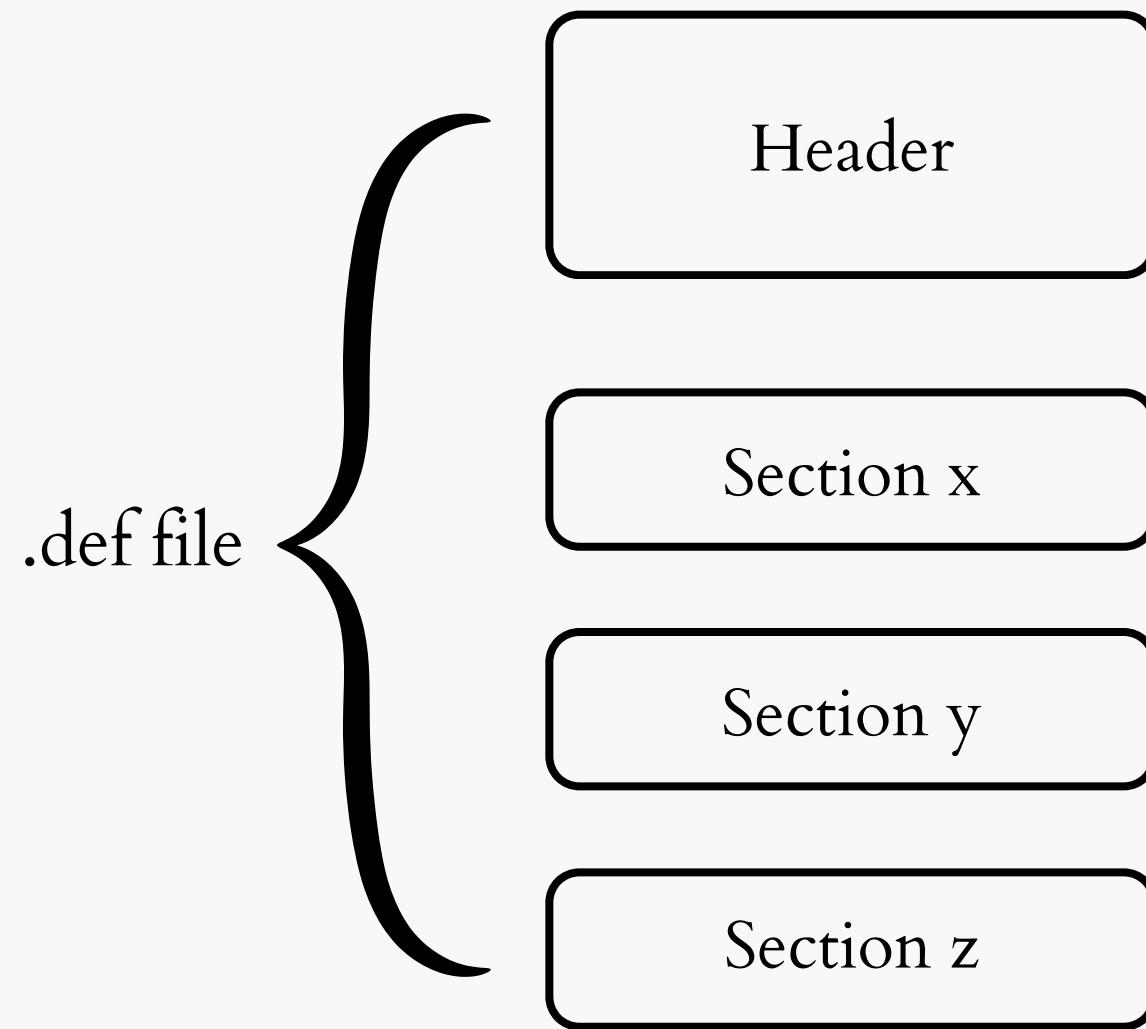
```
Bootstrap: docker
```

```
From: ubuntu:24.04
```

Preferred bootstrap agents:

- docker (images hosted on Docker Hub)
- oras (images from supported OCI registries)
- localimage (images saved on your machine)
- scratch (a flexible option for building a container from scratch)

How to build containers using a .def file



Sections:

Sections are optional. All sections start with % followed by the section name.

Example:

```
%labels
    Author rldubose@ua.edu
    Version v1.0
```

Common sections:

- **%files** – allows you to copy files into the container
- **%post** – where the action happens!
- **%environment** – set environment variables at runtime
- **%runscript** – script executed when the container runs
- **%labels** – metadata
- **%help** – help text for users

Using containers

Apptainer can be used to run your analyses and applications in various ways: running a prepackaged workflow embedded in the image with `apptainer run`, executing commands within the container's environment using `apptainer exec`, or starting an interactive instance to work directly within the container using `apptainer shell`.

- `apptainer run filename.sif`
- `apptainer exec filename.sif <commands>`
 - `apptainer exec --nv filename.sif <commands>` ← Use `--nv` flag for gpu jobs
- `apptainer shell app.sif`

Writing your first container using a .def file

1. `nano helloworld.def`

```
BootStrap: docker
From: ubuntu:24.04

%runscript
    echo "Hello World from HPC Summer Series"
# Prints message when the image is loaded
```

2. Once content pasted, CTRL+O, enter, and CTRL+X

Building and running the container

1. `apptainer build --fakeroot helloworld.sif helloworld.def`

2. `apptainer run helloworld.sif`

NOTE: The fakeroot feature allows an unprivileged user to run a container with the appearance of running as root.



No issues



Help!

Container best practices

1. Do your research on your container sources.

Use trusted resources and inspect the container before use.

2. Ensure that sensitive files like /etc/passwd, /etc/group, and /etc/shadow do not contain secrets.

3. Pay attention to storage.

The default cache directory is ~/.apptainer. It can quickly fill up your storage.

You can change it to scratch:

```
export APPTAINER_CACHEDIR= "$SCRATCH/apptainer-cache"
```

Ask yourself

Does the licensing for the container
fit your needs?

LUNCH BREAK

Please return by 1:30pm

Qualtrics survey

